R3st0

Itération 3
Types de cuisine et livraison

Séance du lundi 2 novembre 2026
4 heures de travail en classe

Vous poursuivez le développement du site de critiques de restaurants. Les visiteurs souhaitent connaître les types de cuisine proposés pour choisir plus facilement un restaurant. Vous devez faire évoluer les données et le site, puis livrer une version stable sur le serveur de votre équipe.

Besoin principal : « En tant que visiteur, je souhaite consulter les types de cuisine de chaque restaurant afin de faciliter mon choix. »

Votre point de départ

Reprenez votre dépôt GitLab et votre dernière version stable. Vérifiez l'état de main, de dev et du site déployé. Conservez les corrections validées. Il peut rester des tickets de l'itération 2. Présentez leur état au Product Owner : terminé et intégré, en cours, bloqué ou reporté. Une difficulté ou un coût élevé ne rend pas un besoin peu important. Faites arbitrer les corrections restantes et les nouveautés dans le même temps disponible.

Un objectif réaliste à convenir

Le backlog dépasse volontairement ce que vous pouvez réaliser en quatre heures. Avant de développer, consultez l'enseignant, qui joue le Product Owner. Proposez un objectif utilisateur, estimez le travail et faites préciser les règles incertaines. Retenez un périmètre qui comprend les tests, la relecture, la mise à jour de la base et le déploiement.



Organiser le backlog et consulter le client

Des récits utilisateurs à transformer en tickets

Une user story exprime un besoin du point de vue de l'utilisateur : « En tant que…, je souhaite…, afin de… ». Une itération peut contenir plusieurs stories. Une story trop grande doit être découpée en résultats vérifiables.

Repère	Besoin proposé	Valeur pour le visiteur
I3-01	Consulter les types de cuisine sur une fiche	Vérifier si un restaurant correspond à ses envies.
I3-02	Consulter les types de cuisine dans la liste des restaurants	Comparer les restaurants sans ouvrir chaque fiche.
I3-03	Consulter les types de cuisine sur l'accueil	Comprendre les propositions mises en avant.
I3-04	Filtrer par un type de cuisine	Trouver des restaurants correspondant à une envie.
I3-05	Combiner plusieurs types de cuisine	Affiner une recherche selon plusieurs préférences.
I3-06	Explorer un répertoire des types de cuisine	Découvrir les choix disponibles et les restaurants associés.

Votre échange initial avec le Product Owner

Présentez les tickets déjà livrés, les blocages, vos estimations S, M ou L et les dépendances. Demandez : quelles pages comptent le plus ? Quel comportement attendre sans type de cuisine renseigné ? Quels besoins reporter si le temps manque ? Quelles règles de recherche faut-il retenir ?

Consignez la réponse et un objectif en une phrase dans docs/iteration3.md. Créez le Milestone « Itération 3 - Types de cuisine », échéance le 2 novembre 2026. Rattachez seulement les Issues retenues ; tracez les reports des anciennes Issues.

Un suivi utile pendant le développement

Chaque Issue comporte le besoin, les critères convenus, la complexité, le responsable, les dépendances, les questions résolues et les tests. Utilisez À faire, En cours, À relire/tester et Terminé. Limitez à un ticket en réalisation par personne et aidez à relire avant d'en démarrer un nouveau.

Désignez un facilitateur dans le rôle de Scrum Master. Il aide l'équipe à se coordonner et à signaler les obstacles. Après le point de cinq minutes, il transmet au Product Owner un rapport bref : ordre retenu, état réel, blocages, questions et tickets encore livrables aujourd'hui avec tests et relecture.

Si un blocage dure plus de quinze minutes, cherchez de l'aide. Si le périmètre doit changer, proposez ce qui sera reporté en échange. Faites confirmer l'arbitrage et datez la décision.

Les besoins de consultation

I3-01 : Consulter les types de cuisine sur une fiche

En tant que visiteur, je souhaite lire les types de cuisine d'un restaurant sur sa fiche afin de savoir s'il correspond à mes envies.

Acceptation : les libellés proviennent de la base, tous les types de cuisine associés apparaissent sans doublon. La fiche fonctionne pour zéro, un ou plusieurs types. Les photos, les avis, la notation et les autres informations déjà présentes restent utilisables.

À clarifier : où placer ces informations, dans quel ordre et quel texte afficher si rien n'est renseigné ? Faites valider un croquis simple avant de modifier la vue. Un simple texte ou des étiquettes lisibles suffisent.

Travail technique associé : examiner les éléments déjà présents dans le modèle et les DAO, concevoir la relation entre restaurants et types de cuisine, puis mettre à jour la base et le parcours MVC. Cette préparation commune sert aussi aux autres tickets ; elle n'est pas à refaire pour chaque page.

I3-02 : Consulter les types de cuisine dans la liste

En tant que visiteur, je souhaite voir les types de cuisine dans la liste des restaurants afin de les comparer rapidement.

Acceptation : chaque restaurant affiche les types de cuisine qui lui correspondent, un restaurant ayant plusieurs types de cuisine n'apparaît pas plusieurs fois dans la liste. Un restaurant sans type de cuisine reste visible. Les liens vers les fiches fonctionnent et les informations concordent avec celles de la fiche.

À clarifier : faut-il afficher tous les types de cuisine sur la carte ou limiter la présentation avec un accès au détail ? Quelles listes sont concernées : liste générale seulement ou aussi résultats de recherche ? Le périmètre retenu doit être explicite.

I3-03 : Consulter les types de cuisine sur l'accueil

En tant que visiteur, je souhaite connaître les types de cuisine des restaurants mis en avant sur l'accueil afin de comprendre les suggestions proposées.

Acceptation : les types de cuisine affichés correspondent aux restaurants présentés. Un restaurant ayant plusieurs types de cuisine n'est pas dupliqué, ses notes ne sont pas faussées par les nouvelles jointures.

À clarifier : même présentation que dans la liste ou version plus compacte ? Si le classement de l'itération 2 est encore incorrect, faites arbitrer sa correction séparément.

Des informations cohérentes entre les pages

Les libellés ne doivent pas être recopiés en dur dans plusieurs vues. Respectez le rôle du modèle, des DAO, des contrôleurs et des vues.

Les besoins de recherche à arbitrer

I3-04 : Filtrer par un type de cuisine

En tant que visiteur, je souhaite choisir un type de cuisine dans la recherche afin d'obtenir les restaurants qui la proposent.

Acceptation : les choix proposés sont chargés depuis la base. Seuls les restaurants associés au type de cuisine sélectionné sont retournés, sans doublon. Sans sélection, aucun filtre de cuisine n'est appliqué. Les critères déjà fonctionnels restent utilisables et la sélection reste visible après la recherche.

À clarifier : dans quel formulaire placer ce filtre ? Comment se combine-t-il avec le nom, la ville et le code postal ? Que montrer si aucun restaurant ne correspond ? Si I2-08 n'est pas terminé, proposez au Product Owner la correction préalable ou un périmètre de filtre autonome explicitement accepté.

Tests : un type avec résultats, un type sans résultat, absence de sélection et combinaison avec un autre critère. Une valeur inexistante ou mal formée doit être gérée côté serveur sans erreur SQL visible.

I3-05 : Combiner plusieurs types de cuisine

En tant que visiteur, je souhaite sélectionner plusieurs types de cuisine afin d'affiner ma recherche selon mes préférences.

Acceptation : plusieurs choix sont possibles, la règle convenue est appliquée, chaque restaurant ne figure qu'une fois, les choix restent visibles, la remise à zéro fonctionne. Une sélection unique et l'absence de sélection restent cohérentes avec I3-04.

À clarifier : Faut-il proposer les restaurants ayant au moins une des cuisines sélectionnées, ou ceux qui les proposent toutes ?

Tests : un restaurant avec seulement le type A, un autre avec seulement B et un troisième avec A et B. Comparez les résultats à la règle validée.

I3-06 : Explorer un répertoire des types de cuisine

En tant que visiteur, je souhaite parcourir les types de cuisine disponibles et connaître le nombre de restaurants associés afin de découvrir les possibilités du site.

Acceptation : une page ou une zone du site présente les libellés et le nombre de restaurants distincts associés à chacun. Un lien permet d'accéder aux restaurants concernés. Les nombres sont calculés à partir des données et concordent avec la liste obtenue.

À clarifier : où accéder à ce répertoire ? Afficher ou masquer les types de cuisine sans restaurant ? Quel ordre de présentation retenir ? Identifiez la dépendance avec le filtrage ou prévoyez un parcours équivalent validé par le Product Owner.

Choisir selon la valeur et le temps disponible

Ces besoins ne sont pas des obligations supplémentaires à réaliser après tous les autres. Ils constituent des possibilités à comparer avec les affichages et les corrections restantes. Une interface d'administration des types de cuisine n'est pas demandée : les données peuvent être préparées par un script SQL.

Un besoin ambitieux peut être reporté. Expliquez ce choix au Product Owner et conservez une description exploitable pour une prochaine évolution.

Faire évoluer les données et vérifier le résultat

Le référentiel des cuisines

1 : sud ouest ; 2 : japonaise ; 3 : orientale ; 4 : fastfood ; 5 : vegetarienne ; 6 : vegan ; 7 : crepe ; 8 : sandwich ; 9 : tartes ; 10 : viande ; 11 : grillade.

Les règles de gestion à respecter

Un restaurant peut proposer zéro, un ou plusieurs types de cuisine. Un type peut concerner plusieurs restaurants. Une même association ne doit pas être enregistrée deux fois et doit référencer un restaurant et un type existants. Une absence d'information ne signifie pas qu'un restaurant ne propose aucune cuisine.

Examinez les classes, méthodes et tables existantes avant de créer de nouveaux éléments. Justifiez les clés et les relations de votre modèle. Documentez sous forme de modèle relationnel.

Une mise à jour reproductible

Versionnez le script de migration, par exemple sql/iteration3_migration.sql, et séparez si utile les données de démonstration. Préservez les restaurants, utilisateurs, critiques et favoris existants. Testez sur une copie de votre base avant toute mise à jour du serveur, ne remplacez pas la base par l'import initial.

Documentez les préconditions, l'ordre des scripts, les contrôles de réussite et le comportement en cas de relance. Faites vérifier la procédure sur la base d'un équipier. Conservez un schéma des relations dans docs/iteration3.md.

Intégrer déployer et présenter la livraison

Un cycle Git par ticket

Partez de dev à jour pour créer une branche de ticket. Faites des commits explicites et ouvrez une Merge Request vers dev avec le numéro de l'Issue, par exemple Refs #23. Avant la relecture finale, récupérez dev et fusionnez sa version à jour dans votre branche, résolvez les conflits puis retestez.

Un équipier relit le code et exécute les tests. Intégrez progressivement les MR validées, puis vérifiez le résultat sur dev. Fermez l'Issue lorsque les critères convenus sont satisfaits et l'acceptation obtenue. Un travail incomplet reste dans une branche et une MR en brouillon.

Préparer une version identifiable

Après les contrôles, ouvrez la MR de dev vers main. Conservez dev et ne faites pas de squash de cette branche durable. Créez le tag v0.3.0 sur le commit validé de main et une Release présentant les changements et limites. Si ce tag existe déjà, convenez d'un nouveau numéro sans déplacer le tag existant.

Déployer sur le serveur de votre équipe

Vérifiez la base et les paramètres propres au serveur ; adaptez le nom de base à votre compte si nécessaire. Aucun identifiant personnel, mot de passe serveur ou export contenant des secrets ne doit être publié dans Git.

Avant modification, sauvegardez la base distante et la version du code actuellement déployée. Appliquez la migration et transférez le code de la version validée. Conservez les sauvegardes hors du répertoire accessible au public. Prévoyez comment restaurer ensemble code et données si la mise à jour échoue.

Vérifiez depuis l'URL du serveur les pages modifiées et les parcours essentiels. Une réussite en local ne suffit pas à déclarer le déploiement réussi. Signalez immédiatement un problème d'accès ou de configuration, avec le message d'erreur et les vérifications effectuées.

Les livrables attendus à 18h

Support	Contenu attendu
GitLab	Issues et tableau à jour, décisions datées, MR relues, tag et Release de la version livrée.
README	Installation et mise à jour, scripts SQL, configuration à adapter et procédure de déploiement.
docs/iteration3.md	Objectif, périmètre retenu, modèle de données, tests, contributions, reports et difficultés.
Rapport au Product Owner	URL du site, version déployée, tickets livrés ou reportés, résultats des tests et limites connues.